DP-2026-001-06-single-dynamics-kernel

One deterministic, dependency-free dynamics kernel serving simulation, training, and control

> **Defensive publication.** This document is published solely to place its subject matter into the public domain of technical knowledge as prior art. **No patent protection is sought by the author for any subject matter described here.**
>
> **Author / declarant:** Jun Kawasaki (`root@junkawasaki.com`) **Disclosure set:** DP-2026-001, part 6 of 10 · first published 2026-08-19 (UTC) **Enabling reference implementation:** public source code under `https://github.com/kotoba-lang/`, licensed Apache-2.0, fixed by commit and tree SHA-1 in the evidence bundle at `https://github.com/com-junkawasaki/prior-art` — that bundle carries RFC 3161 time-stamp tokens and OpenTimestamps attestations over the fixing manifest.

Note on the "Variants and generalizations" section below

The variants are stated deliberately and at length. They are disclosed so that a later filing directed to an obvious variation of the mechanism is met by an express **written** disclosure of that variation, and not merely by an argument that the variation would have been obvious. Where the text says "any", the word is intended in its ordinary broad sense and the enumeration following it is illustrative, not exhaustive.

Working source code implementing the mechanism is publicly available at the commits fixed in the evidence bundle. A person of ordinary skill in robotics software can read, build and run that code; this document is therefore an enabling disclosure and not a mere statement of a result.

Problem

Robotics practice maintains at least three implementations of the same physics: one inside the simulator, one inside the training environment, and one inside the controller's model. They disagree, and the disagreement is discovered on the real robot. The simulators are additionally platform-bound: they cannot run in a browser, in a serverless worker, or inside a verifier.

Mechanism

A single **pure, deterministic, zero-dependency** articulated-body dynamics kernel, written in a portable source language and compiled to every target that needs it (server runtime, browser via WebAssembly, native ahead-of-time binary, and a reference interpreter used as an oracle), such that the same code answers all three roles.

The kernel implements, on spatial-vector (Plücker) algebra:

- rigid-body and articulated-body state, with joint transforms and motion subspaces per joint type;
- forward kinematics to world poses, and the **geometric Jacobian** per link;

- the **recursive Newton-Euler** algorithm for inverse dynamics and bias forces, and the **composite rigid-body** algorithm for the mass matrix, with a factorization-based linear solve;
- **forward dynamics** and gravity-compensation torques;
- contact detection by **GJK** distance and **EPA** penetration on convex shapes, oriented-bounding-box separating-axis tests with manifold generation, sphere–plane and conservative-advancement time-of-impact;
- contact resolution by impulses with an effective-mass computation, friction via a tangent basis, warm starting, and Baumgarte stabilization; joint-limit resolution by the same impulse machinery;
- **inverse kinematics** by damped-least-squares on position and on full pose, with an SO(3) orientation error;
- an **LQR** controller obtained by linearizing about an operating point and solving the discrete algebraic Riccati equation;
- trajectory generation: cubic, quintic, minimum-jerk, and waypoint-sequenced;
- a **vectorized batch** stepping interface with per-environment configuration, for training.

Because the kernel is pure and has no dependencies, its determinism is testable by identity: the same inputs produce byte-identical outputs on every target, and a **parity harness** compares targets against the reference interpreter. Because it has no I/O, it can be embedded inside the governor of Section 1 — a governor can *predict* the consequence of a proposed action using the same physics the simulator used, before admitting it.

Variants and generalizations

- The portable source language may be any language with multiple compilation targets; the targets may include native machine code, WebAssembly, JavaScript, a bytecode, or an interpreted reference form. The claim is the *single-kernel, multi-target, parity-checked* arrangement, not a particular language.
- The dynamics formulation may be: spatial-vector recursive algorithms as above; maximal-coordinate with constraints; reduced-coordinate Lagrangian; articulated-body algorithm; featherstone variants; a differentiable formulation carrying gradients.
- Contact may be resolved by: sequential impulses; projected Gauss-Seidel; a linear- or nonlinear-complementarity-problem solver; compliant/penalty contact; position-based dynamics; an implicit time-stepping scheme.
- The controller may be: LQR; PD with gravity compensation; computed torque; model-predictive control; a learned policy; a hybrid where the learned policy proposes and a model-based term bounds.
- Determinism may be secured by: fixed-point arithmetic; a specified floating-point evaluation order with no fast-math reassociation; a specified reduction order in batch operations; a recorded seed for every stochastic element; byte-comparison against a reference target.
- The three roles unified may be any subset or superset of: simulation, training-environment stepping, controller internal model, governor's predictive check, digital twin, hardware-in-the-loop rig, post-hoc accident reconstruction, and a certification artifact.
- The kernel may be shipped as: a library; a WebAssembly module with a declared import set (Section 2); a native shared object; a service. Where shipped as a capability-gated module, it has

no ambient authority and its outputs are values, so it cannot actuate by itself.

Reference implementation

`kotoba-lang/com-nvidia-isaac-sim` (spatial algebra `plucker/crm/crf/spatial-inertia`; `mass-matrix`, `forward-dynamics`, `inverse-dynamics`, `gravity-torque`, `geometric-jacobian`, `point-jacobian`; `gjk-distance`, `epa-penetration`, `obb-sat`, `obb-manifold`, `conservative-advancement-toi`; `resolve-contacts`, `resolve-static-contact-friction`, `warm-start-static-contact`, `resolve-limits`; `position-ik`, `pose-ik`, `solve-dls`; `solve-dare`, `compute-gain`; `cubic-polynomial-trajectory`, `quintic-polynomial-trajectory`, `min-jerk`, `waypoint-trajectory`; `step-vectorized`, `step-vectorized-per-env`, `set-pd-drive`), `kotoba-lang/physics`, `kotoba-lang/physics-2d`, `kotoba-lang/kami-engine`.

中文摘要 / Chinese abstract

本文件为**防御性公开**（defensive publication）。作者自愿公开以下技术方案，使其成为**现有技术**（专利法第二十二条所称"为公众所知的技术"），并声明**不为其中任何技术方案申请专利**。参考实现的源代码自 2026 年 8 月 19 日起在互联网上公开获取，采用 Apache-2.0 许可，仓库地址与提交哈希见 `https://github.com/com-junkawasaki/prior-art`。

一个确定性、无依赖的动力学内核同时服务于仿真、训练与控制：基于空间向量（Plücker）代数实现正运动学、几何雅可比、递归牛顿-欧拉逆动力学、复合刚体质量矩阵、正动力学与重力补偿；GJK 距离与 EPA 穿透、有向包围盒分离轴与流形生成、保守推进碰撞时刻；冲量法接触求解含有效质量、切向摩擦、热启动与 Baumgarte 稳定化，关节限位复用同一机制；阻尼最小二乘位置与位姿逆运动学；线性化后解离散代数 Riccati 方程得到 LQR；三次、五次、最小抖动与航路点轨迹生成；带逐环境配置的**向量化批处理**步进。该内核为纯函数且无依赖，可编译到原生、WebAssembly 与参考解释器等多目标并以**一致性校验**逐字节比对，因此可嵌入监管器中，在准许某动作之前用与仿真相同的物理**预测其后果**。

上文"变型与推广"一节系**有意穷举列举**，以使针对本机制之显而易见变化的在后申请面对的是明确的书面公开，而非仅仅是"显而易见"的主张。

Statement of dedication